

Contents

STM32F407 Smart Room System - Code Organization	2
PROJECT OVERVIEW	2
PIN CONFIGURATION MAP	3
1. RFID MODULE (RC522)	3
Overview	3
Register Definitions	3
Functions	4
2. LCD DISPLAY MODULE	7
Overview	7
Configuration	7
LCD Control Bits	7
Functions	7
3. BUZZER MODULE	10
Overview	10
Frequency Definitions	10
Timer Configuration	10
Functions	10
4. SERVO MOTOR MODULE	11
Overview	11
Timer Configuration	11
Functions	12
5. FAN PWM CONTROL MODULE	12
Overview	12
Timer Configuration	12
Functions	12
6. ACCELEROMETER MODULE (LIS302DL)	14
Overview	14
Register Definitions	14
Functions	14
7. TEMPERATURE SENSOR MODULE (LM35)	15
Overview	15
ADC Configuration	15
Functions	16
8. DATA STRUCTURES	17
MotionData	17
RFIDData	17
FanTimer	17
9. MAIN PROGRAM - COMPLETE CODE	17
Initialization Phase	17
Complete Main Function	18
STATE 2: ROOM OCCUPIED - MONITORING	28
EXIT SEQUENCE	28
10. HELPER FUNCTIONS - COMPLETE CODE	29
int_to_string()	29
float_to_string()	30
temp_to_string()	31

11. SYSTEM CONSTANTS	31
Buzzer Tones	31
Motion Thresholds	31
Temperature Steps	32
Timing Parameters	32
HARDWARE CONNECTIONS SUMMARY	32
COMPILATION & TESTING	32
Build Steps	32
Testing Checklist	33
VIVA QUESTIONS & DETAILED EXPLANATIONS	33
QUESTION 1: RFID Module - Registers, Anti-Collision, and Pin Functions	33
Q1.1: Why So Many Registers in RC522?	33
Q1.2: What is the Anti-Collision Algorithm?	34
Q1.3: RFID Module Pin Functions	36
Q1.4: Complete RFID Communication Example	37
QUESTION 2: I2C LCD Module - Why Only 2 Wires?	37
Q2.1: What is I2C and Why Use It?	37
Q2.2: How Does I2C Work?	38
Q2.3: PCF8574 I2C Expander - Bridging the Gap	39
Q2.4: Why I2C LCD in This Project?	40
Q2.5: Complete I2C LCD Operation	40
QUESTION 3: Why DMA for ADC Instead of Other Options?	41
Q3.1: ADC Data Collection Methods	41
Q3.2: Comparison Table	43
Q3.3: DMA Configuration for This Project	44
Q3.4: Real-World Performance Impact	44
QUESTION 4: Communication Protocols Used - General Explanation	45
Q4.1: Protocol Overview Table	45
Q4.2: Detailed Protocol Explanations	46
Q4.3: Protocol Comparison - When to Use Which?	51
Q4.4: Protocol Layering in This System	52
Q4.5: Clock Speed Requirements	52

STM32F407 Smart Room System - Code Organization

PROJECT OVERVIEW

This is a comprehensive smart room automation system for the STM32F407 microcontroller featuring: - **RFID Access Control** (RC522 module via SPI2) - **Temperature Monitoring** (LM35 sensor via ADC) - **LCD Display** (16x2 I2C display) - **Motion Detection** (LIS302DL accelerometer via SPI1) - **Fan Speed Control** (PWM on PD15/TIM4) - **Door Lock** (Servo on PB0/TIM3) - **Buzzer Alerts** (PWM on PA15/TIM2) - **Light Control** (GPIO on PC9) - **Entry/Exit Buttons & IR Sensor** (GPIO inputs)

PIN CONFIGURATION MAP

PC7 → IR Sensor
PC8 → Exit Button
PC9 → LED Light
PE6 → LED OFF Button

PWM OUTPUTS:

PA15 → TIM2_CH1 (Buzzer)
PB0 → TIM3_CH3 (Servo)
PD15 → TIM4_CH4 (Fan)

ACCELEROMETER (SPI1):

PA5 → SCK
PA6 → MISO
PA7 → MOSI
PE3 → CS

LCD (I2C1):

PB6 → SCL
PB7 → SDA

RFID (SPI2):

PB13 → SCK
PB14 → MISO
PB15 → MOSI
PD8 → CS
PD9 → RST
PC6 → IRQ

TEMPERATURE SENSOR:

PA0 → ADC1_IN0 (LM35)

1. RFID MODULE (RC522)

Overview

RC522 RFID module communicates via SPI2 for card detection and UID reading. Supports 13.56 MHz RFID protocol with anti-collision algorithm.

Register Definitions

```
#define CommandReg    0x01
#define ComIrqReg     0x04
```

```

#define FIFODataReg      0x09
#define FIFOLevelReg     0x0A
#define ControlReg       0x0C
#define BitFramingReg    0x0D
#define ErrorReg         0x06
#define ModeReg          0x11
#define TxASKReg         0x15
#define TxControlReg     0x14
#define VersionReg       0x37

#define PCD_IDLE         0x00
#define PCD_TRANSCEIVE   0x0C
#define PCD_RESETPHASE   0x0F

#define PICC_REQIDL      0x26
#define PICC_ANTICOLL    0x93

```

Functions

MFRC522_WriteReg() **Purpose:** Write data to RC522 registers via SPI2

Parameters: - addr: Register address - val: Value to write

Implementation: Shifts address, transmits through SPI2 with CS control

```

void MFRC522_WriteReg(uint8_t addr, uint8_t val)
{
    uint8_t data[2];
    data[0] = (addr << 1) & 0x7E; /* Address format: AAAAAAAAA0 (write bit) */
    data[1] = val;                /* Data to write */

    RFID_CS_LOW();                /* Select RFID module */
    HAL_SPI_Transmit(&hspi2, data, 2, HAL_MAX_DELAY); /* Send address + data */
    RFID_CS_HIGH();              /* Deselect RFID module */
}

```

MFRC522_ReadReg() **Purpose:** Read data from RC522 registers

Parameters: addr - Register address

Returns: Register value (uint8_t)

Implementation: Transmits read command with address, receives response

```

uint8_t MFRC522_ReadReg(uint8_t addr)
{
    uint8_t tx = ((addr << 1) & 0x7E) | 0x80; /* Address format: AAAAAAAAA1 (read bit) */
    uint8_t rx;

    RFID_CS_LOW();                /* Select RFID module */
    HAL_SPI_Transmit(&hspi2, &tx, 1, HAL_MAX_DELAY); /* Send address */
    HAL_SPI_Receive(&hspi2, &rx, 1, HAL_MAX_DELAY); /* Receive data */
}

```

```

RFID_CS_HIGH();                                /* Deselect RFID module */

return rx;
}

```

MFRC522_Init() **Purpose:** Initialize RC522 module, set frequencies and modes

Actions: - Enable oscillator - Set TX gain - Configure timing - Enable TX output

```

void MFRC522_Init(void)
{
    RFID_RST_HIGH();                            /* Normal operation mode */
    HAL_Delay(50);                              /* Wait for chip stabilization */

    MFRC522_WriteReg(CommandReg, PCD_RESETPHASE); /* Reset all registers */
    MFRC522_WriteReg(TxASKReg, 0x40);           /* Configure TX gain */
    MFRC522_WriteReg(ModeReg, 0x3D);            /* Set transmission parameters */
    MFRC522_WriteReg(TxControlReg, 0x83);       /* Enable TX1 and TX2 outputs */
}

```

MFRC522_Request() **Purpose:** Send REQA command to detect card presence

Parameters: tagType[] - Storage for card type

Returns: 1 if card detected, 0 otherwise

Algorithm: 1. Clear FIFO and interrupts 2. Send 7-bit REQA command 3. Wait for IRQ with 1000ms timeout 4. Check error register 5. Read 2-byte ATQA response

```

uint8_t MFRC522_Request(uint8_t *tagType)
{
    /* Clear interrupts and FIFO */
    MFRC522_WriteReg(CommandReg, PCD_IDLE);
    MFRC522_WriteReg(ComIrqReg, 0x7F);          /* Clear all interrupt flags */
    MFRC522_WriteReg(FIFOLevelReg, 0x80);        /* Clear FIFO */

    /* REQA is 7-bit, set TxLastBits=7 and StartSend bit */
    MFRC522_WriteReg(BitFramingReg, 0x07);      /* 7-bit transmission */
    MFRC522_WriteReg(FIFODataReg, PICC_REQIDL); /* Load REQA command (0x26) */
    MFRC522_WriteReg(CommandReg, PCD_TRANSCEIVE); /* Execute transceive */
    MFRC522_WriteReg(BitFramingReg, 0x87);      /* Start transmission */

    /* Wait for the IRQ or timeout */
    uint16_t n = 0;
    while (n < 1000) {
        uint8_t irq = MFRC522_ReadReg(ComIrqReg);
        if (irq & 0x30) break;                  /* RxIRq or TxIRq set */
        n++;
        HAL_Delay(1);                           /* 1ms per iteration */
    }
}

```

```

    /* Stop sending */
    MFRC522_WriteReg(BitFramingReg, 0x07);

    if (n == 1000) return 0; /* Timeout */

    /* Check errors */
    uint8_t err = MFRC522_ReadReg(ErrorReg);
    if (err & 0x1B) return 0; /* Protocol/buffer/CRC error */

    /* Ensure we got ATQA (2 bytes) */
    uint8_t fifo_len = MFRC522_ReadReg(FIFOLevelReg);
    if (fifo_len < 2) return 0;

    tagType[0] = MFRC522_ReadReg(FIFODataReg);
    tagType[1] = MFRC522_ReadReg(FIFODataReg);
    return 1;
}

```

MFRC522_Anticoll() **Purpose:** Read unique UID from detected card

Parameters: uid[] - Storage for 5-byte UID

Returns: 1 if successful, 0 on error

Algorithm: 1. Send anti-collision command 2. Wait for IRQ response 3. Validate last bits (must be 0) 4. Read 5 bytes (4 UID + 1 checksum)

```

uint8_t MFRC522_Anticoll(uint8_t *uid)
{
    MFRC522_WriteReg(CommandReg, PCD_IDLE);
    MFRC522_WriteReg(ComIrqReg, 0x7F); /* Clear interrupt flags */
    MFRC522_WriteReg(FIFOLevelReg, 0x80); /* Clear FIFO */

    MFRC522_WriteReg(BitFramingReg, 0x00);
    MFRC522_WriteReg(FIFODataReg, PICC_ANTICOLL); /* Anti-collision command (0x93) */
    MFRC522_WriteReg(FIFODataReg, 0x20); /* Select all 40 bits */
    MFRC522_WriteReg(CommandReg, PCD_TRANSCEIVE); /* Execute */
    MFRC522_WriteReg(BitFramingReg, 0x80); /* Start transmission */

    uint16_t n = 0;
    while (n < 1000) {
        uint8_t irq = MFRC522_ReadReg(ComIrqReg);
        if (irq & 0x30) break;
        n++;
        HAL_Delay(1);
    }

    MFRC522_WriteReg(BitFramingReg, 0x00);
    if (n == 1000) return 0; /* Timeout */
}

```

```

uint8_t err = MFRC522_ReadReg(ErrorReg);
if (err & 0x1B) return 0;           /* Error detected */

/* Check if last bits received correctly (must be 0) */
uint8_t rx_last_bits = MFRC522_ReadReg(ControlReg) & 0x07;
if (rx_last_bits != 0) return 0;

uint8_t fifo_len = MFRC522_ReadReg(FIFOLevelReg);
if (fifo_len < 5) return 0;        /* Need 5 bytes (4 UID + 1 checksum) */

/* Read UID (5 bytes) */
for (int i = 0; i < 5; i++)
    uid[i] = MFRC522_ReadReg(FIFODataReg);

return 1;
}

```

RFID_Init() (Alias) Purpose: Compatibility wrapper
Action: Calls MFRC522_Init()

2. LCD DISPLAY MODULE

Overview

PCF8574 I2C LCD controller (16x2 character display). I2C address 0x27, 4-bit mode operation.

Configuration

```
#define LCD_ADDR (0x27 << 1)  /* 0x4E with 7-bit left shift */
```

LCD Control Bits

Bit 7-4: Data (D7-D4)
 Bit 3: Enable (EN)
 Bit 2: Read/Write (RS) - 0=Command, 1=Data
 Bit 1: Backlight Enable
 Bit 0: Reserved

Functions

lcd_send_cmd() **Purpose:** Send command to LCD in 4-bit mode
Parameters: cmd - Command byte (e.g., 0x01 = Clear)

Algorithm: 1. Split command into high (D7-D4) and low (D3-D0) nibbles 2. Set RS=0 (command mode), EN=1, send high nibble 3. Toggle EN to latch data 4. Send low nibble with same procedure 5. Transmit 4-byte sequence via I2C

Common Commands: - 0x01: Clear display - 0x02: Return cursor home - 0x28: 4-bit mode, 2 lines - 0x0C: Display ON, cursor OFF - 0x06: Auto-increment cursor - 0x80+col: Move to row 0 - 0xC0+col: Move to row 1

```
void lcd_send_cmd(uint8_t cmd)
{
    uint8_t data_u = (cmd & 0xF0);           /* High nibble (D7-D4) */
    uint8_t data_l = ((cmd << 4) & 0xF0);    /* Low nibble shifted to D7-D4 */

    /* 4-byte sequence for high nibble then low nibble */
    uint8_t data_arr[4] = {
        data_u | 0x0C, /* EN=1, RS=0 (Command), Backlight=1 */
        data_u | 0x08, /* EN=0 (latch data) */
        data_l | 0x0C, /* EN=1 for low nibble */
        data_l | 0x08  /* EN=0 (latch data) */
    };

    HAL_I2C_Master_Transmit(&hi2c1, LCD_ADDR, data_arr, 4, 100);
}
```

lcd_send_data() Purpose: Send character data to LCD

Parameters: data - ASCII character code

Difference from cmd: Sets RS=1 (data mode) instead of RS=0

```
void lcd_send_data(uint8_t data)
{
    uint8_t data_u = (data & 0xF0);           /* High nibble (D7-D4) */
    uint8_t data_l = ((data << 4) & 0xF0);    /* Low nibble shifted to D7-D4 */

    /* 4-byte sequence - same as cmd but with RS=1 (data mode) */
    uint8_t data_arr[4] = {
        data_u | 0x0D, /* EN=1, RS=1 (Data), Backlight=1 */
        data_u | 0x09, /* EN=0 (latch data) */
        data_l | 0x0D, /* EN=1 for low nibble */
        data_l | 0x09  /* EN=0 (latch data) */
    };

    HAL_I2C_Master_Transmit(&hi2c1, LCD_ADDR, data_arr, 4, 100);
}
```

lcd_init() Purpose: Initialize LCD with 4-bit mode sequence

Sequence:

1. Wait 50ms (power stabilization)

2. 0x30 - Set 8-bit mode
3. 0x02 - Return home
4. 0x28 - Configure 4-bit, 2 lines, 5x8 font
5. 0x0C - Turn display ON
6. 0x06 - Set increment mode
7. 0x01 - Clear display
8. Wait 2ms

```
void lcd_init(void)
{
    HAL_Delay(50);                /* Power stabilization */

    lcd_send_cmd(0x30);           /* 8-bit mode initialization */
    lcd_send_cmd(0x02);           /* Return home */
    lcd_send_cmd(0x28);           /* 4-bit mode, 2 lines, 5x8 font */
    lcd_send_cmd(0x0C);           /* Display ON, cursor OFF, blink OFF */
    lcd_send_cmd(0x06);           /* Increment mode, no shift */
    lcd_send_cmd(0x01);           /* Clear display */

    HAL_Delay(2);                 /* Wait for clear to complete */
}
```

lcd_clear() **Purpose:** Clear all display content

Action: Send 0x01 command, wait 2ms for execution

```
void lcd_clear(void)
{
    lcd_send_cmd(0x01);           /* Clear display command */
    HAL_Delay(2);                 /* Wait for clear operation */
}
```

lcd_print() **Purpose:** Print null-terminated string to LCD

Parameters: str - Pointer to string

Algorithm:

```
while (*str != '\0') {
    lcd_send_data(*str++)
}

void lcd_print(char *str)
{
    while (*str)
        lcd_send_data(*str++);    /* Send each character until null terminator */
}
```

lcd_set_cursor() **Purpose:** Move cursor to specific position

Parameters: - row: 0 (first line) or 1 (second line) - col: 0-15 (column position)

Calculation: - Row 0: Address = 0x80 + col - Row 1: Address = 0xC0 + col

Formula: Sends DDRAM address command

```
void lcd_set_cursor(uint8_t row, uint8_t col)
{
    uint8_t address = (row == 0) ? (0x80 + col) : (0xC0 + col);
    lcd_send_cmd(address);
}
```

3. BUZZER MODULE

Overview

Buzzer controlled via PWM on PA15 (TIM2_CH1). Frequency varies from 165Hz to 1047Hz for different alerts.

Frequency Definitions

```
#define NOTE_E4          330    /* Neutral tone (Entry/Exit) */
#define NOTE_AFFIRMATIVE 523    /* Success confirmation (C5) */
#define NOTE_ALERT       165    /* Warning/Failure (E3) */
#define NOTE_EARTHQUAKE 1047    /* Critical alarm (C6) */
```

Timer Configuration

Clock: 1 MHz (84MHz / 84 prescaler)
Period: Dynamic (calculated from frequency)
Frequency = 1,000,000 / (Period + 1)

Functions

buzzer_play() **Purpose:** Generate sound at specified frequency

Parameters: freq - Frequency in Hz (165-1047)

Algorithm: 1. Calculate period: (1MHz / freq) - 1 2. Stop any existing PWM 3. Wait 5ms for clean stop 4. Disable/reset timer counter 5. Update auto-reload register 6. Set compare (50% duty cycle for square wave) 7. Start PWM

Frequency Examples: - 330 Hz (NOTE_E4): Period = 3030 - 523 Hz (NOTE_AFFIRMATIVE): Period = 1911 - 165 Hz (NOTE_ALERT): Period = 6060 - 1047 Hz (NOTE_EARTHQUAKE): Period = 954

```
void buzzer_play(uint32_t freq)
{
    uint32_t timer_clock = 1000000;          /* Timer clock = 1 MHz */
    uint32_t period = (timer_clock / freq) - 1; /* Calculate period */

    /* CRITICAL: Stop timer before changing parameters */
}
```

```

HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_1);
HAL_Delay(5);                                /* Brief delay to ensure timer stops */

/* Reset the timer counter to ensure clean restart */
__HAL_TIM_DISABLE(&htim2);
htim2.Instance->CNT = 0;                      /* Reset counter */
__HAL_TIM_ENABLE(&htim2);

/* Update timer parameters */
__HAL_TIM_SET_AUTORELOAD(&htim2, period);
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, period / 2); /* 50% duty */

/* Start PWM */
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
}

```

buzzer_stop() Purpose: Stop buzzer output

Action: Disable PWM on TIM2_CH1

```

void buzzer_stop(void)
{
    HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_1);
}

```

4. SERVO MOTOR MODULE

Overview

Standard servo (0-180°) controlled via PWM on PB0 (TIM3_CH3). Uses 50Hz base frequency with 1-2ms pulse width.

Timer Configuration

Clock: 1 MHz

Frequency: 50 Hz (20ms period)

Pulse Range: 1000-2000 µs (1-2 ms)

Angle Calculation:

0°: 1000 µs

90°: 1500 µs (neutral)

180°: 2000 µs

Pulse = 1000 + (angle * 1000 / 180)

Functions

servo_set_angle() **Purpose:** Move servo to specific angle

Parameters: angle - Desired angle (0-180°)

Algorithm: 1. Clamp angle to 0-180 range 2. Calculate pulse width: $1000 + (\text{angle} * 1000 / 180)$ 3. Update PWM compare register 4. Servo moves to position

Usage Examples: - 0°: Locked (door closed) - 90°: Neutral/Half-open - 180°: Unlocked (door open)

```
void servo_set_angle(uint8_t angle)
{
    if(angle > 180) angle = 180;           /* Clamp to max */

    /* Calculate pulse width: 1000-2000 μs */
    uint32_t pulse = 1000 + ((uint32_t)angle * 1000) / 180;

    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, pulse);
}
```

5. FAN PWM CONTROL MODULE

Overview

3-wire brushless fan controlled via PWM on PD15 (TIM4_CH4). Speed ramps from 0-100% based on temperature.

Timer Configuration

Clock: 1 MHz

Frequency: 50 Hz (20ms period)

Period: 20000 counts

Duty Cycle = $(\text{Pulse} / 20000) \times 100\%$

Speed Levels:

0%: Pulse = 0

25%: Pulse = 5000

50%: Pulse = 10000

75%: Pulse = 15000

100%: Pulse = 20000

Functions

fan_start() **Purpose:** Enable PWM output to fan

Action: Start TIM4_CH4 PWM, set running flag

```
void fan_start(void)
{
```

```

    HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_4);
    fan_timer.running = 1;                                /* Mark fan as running */
}

```

fan_set_speed() **Purpose:** Set fan speed via PWM duty cycle

Parameters: duty_cycle - 0-20000 (0-100%)

Algorithm: 1. Clamp to max (20000) 2. Update PWM compare register 3. Fan speed adjusts immediately

```

void fan_set_speed(uint16_t duty_cycle)
{
    /* Clamp duty_cycle to max period (20000) */
    if (duty_cycle > 20000) duty_cycle = 20000;

    /* Set PWM pulse width */
    __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_4, duty_cycle);
}

```

fan_stop() **Purpose:** Stop fan and disable PWM

Action: Reinitialize TIM4 (resets all parameters)

```

void fan_stop(void)
{
    MX_TIM4_Init();                                     /* Reinitialize to reset all settings */
}

```

fan_set_duration() **Purpose:** Run fan for fixed time at specific speed

Parameters: - duration_ms: Auto-stop time (0 = no auto-stop) - speed: PWM value (0-20000)

Action: Starts fan, records start time, enables auto-stop flag

```

void fan_set_duration(uint16_t duration_ms, uint16_t speed)
{
    fan_start();                                       /* Enable PWM */
    fan_set_speed(speed);                             /* Set speed */
    fan_timer.duration_ms = duration_ms;              /* Record duration */
    fan_timer.start_time = HAL_GetTick();             /* Record start time */
    fan_timer.running = 1;
}

```

fan_update_timer() **Purpose:** Check and enforce auto-stop duration

Note: **MUST** be called in main loop

Algorithm:

```

if (!running) return
if (duration_ms == 0) return // No auto-stop

```

```

elapsed = current_time - start_time
if (elapsed >= duration_ms)
    fan_set_speed(0)
    fan_stop()

void fan_update_timer(void)
{
    if (!fan_timer.running) return;           /* Fan not running */

    if (fan_timer.duration_ms == 0) return;    /* No auto-stop set */

    uint32_t elapsed = HAL_GetTick() - fan_timer.start_time;

    if (elapsed >= fan_timer.duration_ms) {    /* Timer expired */
        fan_set_speed(0);                     /* Turn off */
        fan_stop();                           /* Disable PWM */
    }
}

```

6. ACCELEROMETER MODULE (LIS302DL)

Overview

3-axis MEMS accelerometer on SPI1 for motion detection and earthquake sensing.

Register Definitions

```

#define LIS302DL_CTRL_REG1 0x20 /* Control register */
#define LIS302DL_OUT_X     0x29 /* X-axis output */
#define LIS302DL_OUT_Y     0x2B /* Y-axis output */
#define LIS302DL_OUT_Z     0x2D /* Z-axis output */

```

Functions

ACC_Init() **Purpose:** Initialize accelerometer with default settings

Action: Write 0x47 to CTRL_REG1 (enables all axes, 400Hz ODR)

```

void ACC_Init(void)
{
    ACC_WriteReg(LIS302DL_CTRL_REG1, 0x47); /* Enable all axes, 400 Hz ODR */
}

```

ACC_WriteReg() **Purpose:** Write value to accelerometer register

Parameters: - reg: Register address - data: Value to write

Algorithm: 1. Set CS low 2. Transmit [register, value] 3. Set CS high

```

void ACC_WriteReg(uint8_t reg, uint8_t data)
{
    uint8_t buf[2] = {reg, data};           /* Create 2-byte command */

    ACC_CS_LOW();                           /* Select accelerometer */
    HAL_SPI_Transmit(&hspi1, buf, 2, HAL_MAX_DELAY); /* Send via SPI1 */
    ACC_CS_HIGH();                          /* Deselect */
}

```

ACC_ReadAxis() **Purpose:** Read single axis output value

Parameters: reg - Register (OUT_X, OUT_Y, or OUT_Z)

Returns: Signed 8-bit axis value

Algorithm: 1. Set address = reg | 0x80 (read bit) 2. Set CS low 3. Transmit address 4. Receive 1 byte 5. Set CS high 6. Return as signed int8_t

```

int8_t ACC_ReadAxis(uint8_t reg)
{
    uint8_t value;
    uint8_t address = reg | 0x80;           /* Set read bit (MSB) */

    ACC_CS_LOW();                           /* Select accelerometer */
    HAL_SPI_Transmit(&hspi1, &address, 1, HAL_MAX_DELAY); /* Send address */
    HAL_SPI_Receive(&hspi1, &value, 1, HAL_MAX_DELAY);    /* Receive data */
    ACC_CS_HIGH();                          /* Deselect */

    return (int8_t)value;                   /* Return as signed value */
}

```

7. TEMPERATURE SENSOR MODULE (LM35)

Overview

Analog temperature sensor on PA0 (ADC1_IN0). Linear output: 10mV per °C.

ADC Configuration

Channel: ADC1_IN0 (PA0)

Resolution: 12-bit (0-4095)

Reference: 3.3V

Sampling: 112 cycles (max accuracy)

DMA: Circular mode (continuous)

Conversion Formula:

Voltage = ADC_Value × (3.3V / 4095)

Temperature (°C) = Voltage × 100

Simplified: Temp = (ADC_Value × 330) / 4095

Functions

LM35_StartConversion() **Purpose:** Begin continuous ADC conversion with DMA
Action: Start ADC1 with DMA circular mode

```
void LM35_StartConversion(void)
{
    HAL_ADC_Start_DMA(&hadc1, (uint32_t *)&adc_value, 1);
    /* DMA now continuously updates adc_value without CPU intervention */
}
```

LM35_ReadADC() **Purpose:** Get raw ADC value from DMA buffer
Returns: 12-bit ADC value (0-4095)

```
uint32_t LM35_ReadADC(void)
{
    return adc_value;          /* Return latest DMA value */
}
```

LM35_GetTemperature() **Purpose:** Convert ADC to temperature with filtering
Returns: Temperature in °C (float)

Algorithm: 1. Read raw ADC from DMA buffer 2. Apply exponential filter: $adc_avg = (adc_avg \times 31 + raw) / 32$ - Slow response (~30ms), very smooth 3. Convert to voltage: $voltage = adc_avg \times 3.3 / 4095$ 4. Convert to temperature: $temp = voltage \times 100$ 5. Return temperature value

Temperature Ranges Used:

20°C: Fan OFF
20-21°C: Fan 25% (5000 PWM)
21-22°C: Fan 50% (10000 PWM)
22-23°C: Fan 75% (15000 PWM)
>23°C: Fan 100% (20000 PWM)

```
float LM35_GetTemperature(void)
{
    /* Get current ADC value from DMA buffer */
    uint32_t raw_adc = LM35_ReadADC();

    /* Smooth exponential filter: 31/32 (slower response ~30ms, very smooth) */
    adc_avg = (adc_avg * 31 + raw_adc) / 32;

    /* Convert ADC value to voltage (0-3.3V) */
    float voltage = (adc_avg * 3.3f) / 4095.0f;

    /* Convert voltage to temperature (LM35: 10mV per °C) */
    float temperature = voltage * 100.0f;
}
```



```

    return temperature;
}

```

8. DATA STRUCTURES

MotionData

```

typedef struct {
    int8_t x, y, z;                /* Current axis values */
    uint16_t motion_count;         /* Motion event counter */
    uint32_t last_motion_time;     /* Last alert timestamp */
    int8_t baseline_x, baseline_y, baseline_z; /* Reference position */
    uint8_t calibrated;           /* Calibration flag */
} MotionData;

```

RFIDData

```

typedef struct {
    uint8_t uid[10];              /* Card UID (5 bytes used) */
    uint8_t uid_len;              /* UID length */
    uint8_t card_detected;        /* Detection flag */
    uint32_t last_detect_time;    /* Last detection time */
} RFIDData;

```

FanTimer

```

typedef struct {
    uint16_t duration_ms;         /* Auto-stop duration */
    uint32_t start_time;          /* Start timestamp */
    uint8_t running;              /* Running flag */
} FanTimer;

```

9. MAIN PROGRAM - COMPLETE CODE

Initialization Phase

1. **System Clock:** 168 MHz
2. **Peripherals:** GPIO, DMA, ADC, SPI1/2, TIM2/3/4, I2C1
3. **Devices:** RFID, Accelerometer, LCD
4. **Calibration:** Accelerometer baseline capture
5. **Door Lock:** Set to 0° (locked)

Complete Main Function

```
int main(void)
{
    HAL_Init();
    SystemClock_Config();

    /* Initialize all peripherals in order */
    MX_GPIO_Init();
    MX_DMA_Init(); /* Enable DMA for ADC */
    MX_ADC1_Init();
    MX_SPI1_Init();
    MX_SPI2_Init();
    MX_TIM2_Init();
    MX_TIM3_Init();
    MX_TIM4_Init();
    MX_I2C1_Init();
    MX_EXTI_Init();

    /* Initialize devices */
    RFID_Init();
    ACC_Init();
    lcd_init();
    LM35_StartConversion(); /* Start ADC */

    /* LCD Welcome Screen */
    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_print("Smart Room");
    lcd_set_cursor(1, 0);
    lcd_print("System Ready");
    HAL_Delay(2000);

    /* Calibrate accelerometer baseline */
    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_print("Calibrating...");
    HAL_Delay(500);
    motion.baseline_x = ACC_ReadAxis(LIS302DL_OUT_X);
    motion.baseline_y = ACC_ReadAxis(LIS302DL_OUT_Y);
    motion.baseline_z = ACC_ReadAxis(LIS302DL_OUT_Z);
    motion.calibrated = 1;
    lcd_set_cursor(1, 0);
    lcd_print("Ready!");
    HAL_Delay(1000);

    /* Lock door initially */
    servo_set_angle(0); /* 0° = Locked */
}
```

```

/* ===== MAIN AUTOMATION LOOP ===== */

uint8_t room_occupied = 0;
uint8_t tagType[2];
uint8_t uid[10];
uint8_t light_manual_control = 0; /* Track if user manually controlled light */
char temp_str[16];
char display_buf[32];
uint8_t lcd_state = 0; /* Sliding display state (0=temp, 1=motion, 2=light) */
uint32_t last_lcd_update = 0;

while(1)
{
    /* ===== STATE 1: WAITING FOR ENTRY ===== */
    if (!room_occupied) {
        lcd_clear();
        lcd_set_cursor(0, 0);
        lcd_print("Scan RFID Card");
        lcd_set_cursor(1, 0);
        lcd_print("To Enter...");

        /* Wait for RFID card */
        if (MFRC522_Request(tagType) == 1) {
            if (MFRC522_Anticoll(uid) == 1) {
                /* Validate against whitelist cards */
                uint8_t valid_card = 0;
                uint8_t card1[] = {0x63, 0x95, 0x4e, 0x56}; /* 63954e56 */
                uint8_t card2[] = {0xd0, 0x6e, 0x6d, 0x32}; /* d06e6d32 */

                /* Check if UID matches whitelist */
                if ((uid[0] == card1[0] && uid[1] == card1[1] &&
                    uid[2] == card1[2] && uid[3] == card1[3]) ||
                    (uid[0] == card2[0] && uid[1] == card2[1] &&
                    uid[2] == card2[2] && uid[3] == card2[3])) {
                    valid_card = 1;
                }

                if (!valid_card) {
                    /* Invalid card - Access Denied */
                    lcd_clear();
                    lcd_set_cursor(0, 0);
                    lcd_print("Access DENIED!");

                    /* Rejection beep (lower tone) */
                    buzzer_play(165); /* NOTE_ALERT */
                    HAL_Delay(300);
                    buzzer_stop();
                }
            }
        }
    }
}

```

```

        HAL_Delay(2000);
    } else {
        /* Valid card detected - Access Granted */
        lcd_clear();
        lcd_set_cursor(0, 0);
        lcd_print("Access Granted!");

        /* Beep confirmation */
        buzzer_play(NOTE_E4);
        HAL_Delay(200);
        buzzer_stop();

        /* Unlock door */
        servo_set_angle(180); /* 180° = Unlocked */
        HAL_Delay(1000);

        /* Wait for person to enter (IR sensor) */
        lcd_clear();
        lcd_set_cursor(0, 0);
        lcd_print("Door Open");
        lcd_set_cursor(1, 0);
        lcd_print("Enter Now...");

        uint32_t wait_start = HAL_GetTick();
        uint32_t last_ir_detect_time = HAL_GetTick();
        uint8_t person_entered = 0;
        uint8_t door_timeout = 0;

        while ((HAL_GetTick() - wait_start) < 15000) {
            /* No motion timeout: close door if no IR detection for 5 seconds */
            uint32_t no_motion_duration = HAL_GetTick() - last_ir_detect_time;
            if (no_motion_duration > 5000) {
                door_timeout = 1;
                break;
            }

            if (ir_sensor_read() == GPIO_PIN_SET) {
                last_ir_detect_time = HAL_GetTick();
                HAL_Delay(2000);

                /* Wait until path is clear */
                uint32_t clear_start = HAL_GetTick();
                while ((HAL_GetTick() - clear_start) < 3000) {
                    if (ir_sensor_read() == GPIO_PIN_RESET) {
                        person_entered = 1;
                        break;
                    }
                }
                HAL_Delay(50);
            }
        }
    }
}

```

```

        }
        break;
    }
    HAL_Delay(50);
}

/* Close door after delay */
HAL_Delay(3000);
servo_set_angle(0); /* Lock door */

if (person_entered) {
    /* PERSON SUCCESSFULLY ENTERED */
    buzzer_play(NOTE_AFFIRMATIVE);
    HAL_Delay(150);
    buzzer_stop();
    HAL_Delay(100);
    buzzer_play(NOTE_AFFIRMATIVE);
    HAL_Delay(150);
    buzzer_stop();

    /* Activate room systems */
    room_occupied = 1;
    light_on();
    light_manual_control = 0;

    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_print("Welcome!");
    HAL_Delay(1500);
} else if (door_timeout) {
    /* NO MOTION DETECTED FOR 5 SEC */
    buzzer_play(NOTE_ALERT);
    HAL_Delay(300);
    buzzer_stop();
    HAL_Delay(100);
    buzzer_play(NOTE_ALERT);
    HAL_Delay(300);
    buzzer_stop();

    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_print("No Motion!");
    lcd_set_cursor(1, 0);
    lcd_print("Door Locked");
    HAL_Delay(2000);
} else {
    /* ENTRY FAILED/TIMEOUT */
    buzzer_play(NOTE_ALERT);

```

```

        HAL_Delay(300);
        buzzer_stop();
        HAL_Delay(100);
        buzzer_play(NOTE_ALERT);
        HAL_Delay(300);
        buzzer_stop();

        lcd_clear();
        lcd_set_cursor(0, 0);
        lcd_print("Entry Failed");
        lcd_set_cursor(1, 0);
        lcd_print("No Detection");
        HAL_Delay(2000);
    }
}

}

HAL_Delay(100);
}

/* ===== STATE 2: ROOM OCCUPIED - MONITORING ===== */
if (room_occupied) {
    /* Read temperature */
    float temp = LM35_GetTemperature();

    /* Fan control: step-based speed ramping */
    uint16_t fan_speed = 0;

    if (temp > 23.0f) {
        fan_speed = 20000; /* 100% */
    } else if (temp > 22.0f) {
        fan_speed = 15000; /* 75% */
    } else if (temp > 21.0f) {
        fan_speed = 10000; /* 50% */
    } else if (temp > 20.0f) {
        fan_speed = 5000; /* 25% */
    } else {
        fan_speed = 0; /* OFF */
    }

    /* Apply fan speed */
    if (fan_speed > 0) {
        if (!fan_timer.running) {
            fan_start();
        }
        fan_set_speed(fan_speed);
    } else {

```

```

        if (fan_timer.running) {
            fan_set_speed(0);
            HAL_Delay(50);
            fan_stop();
        }
    }

    /* Manual light control with PE6 button */
    if (led_off_button_read()) {
        if (HAL_GPIO_ReadPin(GPIOC, LED_LIGHT_PIN) == GPIO_PIN_SET) {
            light_off();
        } else {
            light_on();
        }
        light_manual_control = 1;
        HAL_Delay(300); /* Debounce */
    }

    /* Motion detection with baseline comparison */
    motion.x = ACC_ReadAxis(LIS302DL_OUT_X);
    motion.y = ACC_ReadAxis(LIS302DL_OUT_Y);
    motion.z = ACC_ReadAxis(LIS302DL_OUT_Z);

    /* Calculate delta from baseline */
    int16_t delta_x = abs(motion.x - motion.baseline_x);
    int16_t delta_y = abs(motion.y - motion.baseline_y);
    int16_t delta_z = abs(motion.z - motion.baseline_z);
    int16_t total_delta = delta_x + delta_y + delta_z;

    /* Minor movement detection */
    if (total_delta > 10 && total_delta < 40) {
        uint32_t current_time = HAL_GetTick();
        if (current_time - motion.last_motion_time > 3000) {
            for (int i = 0; i < 3; i++) {
                buzzer_play(NOTE_E4);
                HAL_Delay(80);
                buzzer_stop();
                HAL_Delay(80);
            }
            motion.last_motion_time = current_time;
        }
    }

    /* Earthquake detection */
    if (total_delta > 35) {
        lcd_clear();
        lcd_set_cursor(0, 0);
        lcd_print("*** QUAKE! ***");
    }

```

```

    lcd_set_cursor(1, 0);
    lcd_print("TAKE COVER!");

    for (int i = 0; i < 8; i++) {
        buzzer_play(NOTE_EARTHQUAKE);
        HAL_Delay(150);
        buzzer_stop();
        HAL_Delay(80);
    }

    HAL_Delay(1000);
}

/* Sliding LCD display */
uint32_t current_time = HAL_GetTick();
if (current_time - last_lcd_update > 2000) {
    lcd_state = (lcd_state + 1) % 3;
    last_lcd_update = current_time;

    lcd_clear();

    if (lcd_state == 0) {
        /* Temperature & Fan */
        lcd_set_cursor(0, 0);
        lcd_print("Temp: ");
        temp_to_string(temp, display_buf);
        lcd_print(display_buf);

        lcd_set_cursor(1, 0);
        if (!fan_timer.running) {
            lcd_print("Fan: OFF");
        } else if (temp > 23.0f) {
            lcd_print("Fan: ON 100%");
        } else if (temp > 22.0f) {
            lcd_print("Fan: ON 75%");
        } else if (temp > 21.0f) {
            lcd_print("Fan: ON 50%");
        } else {
            lcd_print("Fan: ON 25%");
        }
    }

    else if (lcd_state == 1) {
        /* Motion Status */
        lcd_set_cursor(0, 0);
        lcd_print("Motion Delta:");

        lcd_set_cursor(1, 0);
        sprintf(display_buf, "%d ", total_delta);
    }
}

```



```

        lcd_print(display_buf);
        if (total_delta > 60) {
            lcd_print("QUAKE!");
        } else if (total_delta > 20) {
            lcd_print("Alert");
        } else {
            lcd_print("Normal");
        }

    } else if (lcd_state == 2) {
        /* Light & System Status */
        lcd_set_cursor(0, 0);
        lcd_print("Light: ");
        if (HAL_GPIO_ReadPin(GPIOC, LED_LIGHT_PIN) == GPIO_PIN_SET) {
            lcd_print("ON");
        } else {
            lcd_print("OFF");
        }

        lcd_set_cursor(1, 0);
        lcd_print("Room: Active");
    }
}

/* Check exit button */
if (exit_button_read()) {
    /* Person wants to exit */
    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_print("Exiting...");
    lcd_set_cursor(1, 0);
    lcd_print("Please Wait");

    /* Unlock door */
    servo_set_angle(180);
    HAL_Delay(3000);

    /* Shutdown all systems */
    light_off();
    if (fan_timer.running) {
        fan_set_speed(0);
        HAL_Delay(100);
        fan_stop();
    }

    /* Wait for person to exit */
    lcd_clear();
    lcd_set_cursor(0, 0);

```

```

lcd_print("Door Open");
lcd_set_cursor(1, 0);
lcd_print("Exit Now...");

uint32_t exit_wait = HAL_GetTick();
uint32_t last_exit_ir_time = HAL_GetTick();
uint8_t person_exited = 0;
uint8_t exit_timeout = 0;

while ((HAL_GetTick() - exit_wait) < 15000) {
    uint32_t exit_no_motion = HAL_GetTick() - last_exit_ir_time;
    if (exit_no_motion > 5000) {
        exit_timeout = 1;
        break;
    }

    if (ir_sensor_read() == GPIO_PIN_SET) {
        last_exit_ir_time = HAL_GetTick();
        HAL_Delay(1000);

        uint32_t exit_clear_start = HAL_GetTick();
        uint8_t path_clear_detected = 0;

        while ((HAL_GetTick() - exit_clear_start) < 5000) {
            if (ir_sensor_read() == GPIO_PIN_RESET) {
                path_clear_detected = 1;
                person_exited = 1;
                break;
            }
            HAL_Delay(50);
        }

        if (!path_clear_detected) {
            person_exited = 1;
        }

        break;
    }
    HAL_Delay(50);
}

/* Close door */
HAL_Delay(3000);
servo_set_angle(0);

if (person_exited) {
    /* PERSON SUCCESSFULLY EXITED */
    buzzer_play(NOTE_AFFIRMATIVE);
}

```

```

    HAL_Delay(150);
    buzzer_stop();
    HAL_Delay(100);
    buzzer_play(NOTE_AFFIRMATIVE);
    HAL_Delay(150);
    buzzer_stop();

    room_occupied = 0;

    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_print("Goodbye!");
    lcd_set_cursor(1, 0);
    lcd_print("Room Secured");
    HAL_Delay(2000);
} else if (exit_timeout) {
    /* NO MOTION - UNDO EXIT */
    buzzer_play(NOTE_ALERT);
    HAL_Delay(300);
    buzzer_stop();
    HAL_Delay(100);
    buzzer_play(NOTE_ALERT);
    HAL_Delay(300);
    buzzer_stop();

    servo_set_angle(0);
    light_on();

    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_print("Exit Cancelled");
    lcd_set_cursor(1, 0);
    lcd_print("Room Re-Active");
    HAL_Delay(2000);
} else {
    /* EXIT FAILED - UNDO EXIT */
    buzzer_play(NOTE_ALERT);
    HAL_Delay(300);
    buzzer_stop();
    HAL_Delay(100);
    buzzer_play(NOTE_ALERT);
    HAL_Delay(300);
    buzzer_stop();

    servo_set_angle(0);
    light_on();

    lcd_clear();

```

```

        lcd_set_cursor(0, 0);
        lcd_print("Exit Failed");
        lcd_set_cursor(1, 0);
        lcd_print("Room Re-Active");
        HAL_Delay(2000);
    }
}

    HAL_Delay(500);
}
}

```

- Display: "Scan RFID Card"
- Wait for valid RFID card
- Valid cards: 0x63954e56, 0xd06e6d32
- On valid: Unlock door (90°), wait for IR sensor
- IR Detection: Person entering detected
- Path Clear: Close door (0°), set room_occupied=1

STATE 2: ROOM OCCUPIED - MONITORING

- **Temperature Control:** Read LM35, adjust fan speed
- **Light Control:** Toggle with PE6 button
- **Motion Detection:** Accelerometer baseline comparison
 - Minor (10-40 delta): 3 beeps every 3 seconds
 - Earthquake (>35 delta): Alert display + 8 beeps
- **LCD Display:** Rotating 3-state display (2 sec each)
 - State 0: Temperature & Fan status
 - State 1: Motion Delta & Status
 - State 2: Light status & System state
- **Exit Handler:** PC8 button triggers exit sequence

EXIT SEQUENCE

- Unlock door (180°)
 - Turn off light
 - Stop fan
 - Wait for IR sensor clear
 - Close door (0°)
 - Set room_occupied=0
-

10. HELPER FUNCTIONS - COMPLETE CODE

int_to_string()

Purpose: Convert integer to string (helper for float conversion)

Parameters: value - Integer, buffer - Output storage

Returns: String length

```
static uint8_t int_to_string(int32_t value, char *buffer)
{
    uint8_t i = 0;
    uint8_t is_negative = 0;

    /* Handle negative numbers */
    if (value < 0) {
        is_negative = 1;
        value = -value;
    }

    /* Handle zero case */
    if (value == 0) {
        buffer[i++] = '0';
        buffer[i] = '\0';
        return i;
    }

    /* Convert digits (reverse order) */
    uint8_t start = i;
    while (value > 0) {
        buffer[i++] = '0' + (value % 10);
        value /= 10;
    }

    /* Add negative sign */
    if (is_negative) {
        buffer[i++] = '-';
    }

    /* Reverse the string */
    uint8_t end = i - 1;
    start = is_negative ? 1 : 0;
    while (start < end) {
        char temp = buffer[start];
        buffer[start] = buffer[end];
        buffer[end] = temp;
        start++;
        end--;
    }
}
```

```

    buffer[i] = '\\0';
    return i;
}

```

float_to_string()

Purpose: Convert float to string with decimals

Parameters: value - Float, buffer - Output, decimals - Places (0-2)

```

static void float_to_string(float value, char *buffer, uint8_t decimals)
{
    /* Handle negative values */
    uint8_t is_negative = 0;
    if (value < 0) {
        is_negative = 1;
        value = -value;
        *buffer++ = '-';
    }

    /* Extract integer part */
    int32_t int_part = (int32_t)value;

    /* Extract fractional part */
    float frac_part = value - (float)int_part;

    /* Convert integer part */
    uint8_t len = int_to_string(int_part, buffer);
    buffer += len;

    /* Add decimal point and fractional part */
    if (decimals > 0) {
        *buffer++ = '.';

        /* Scale fractional part based on decimal places */
        if (decimals == 1) {
            frac_part *= 10.0f;
        } else if (decimals == 2) {
            frac_part *= 100.0f;
        }

        int32_t frac_int = (int32_t)(frac_part + 0.5f); /* Round */

        /* Handle carry-over from rounding */
        if (decimals == 1 && frac_int >= 10) {
            frac_int = 0;
        } else if (decimals == 2 && frac_int >= 100) {
            frac_int = 0;
        }
    }
}

```

```

    /* Convert fractional digits */
    if (decimals == 1) {
        *buffer++ = '0' + (frac_int % 10);
    } else if (decimals == 2) {
        *buffer++ = '0' + ((frac_int / 10) % 10);
        *buffer++ = '0' + (frac_int % 10);
    }
}

*buffer = '\0';
}

```

temp_to_string()

Purpose: Convert temperature to “##.#C” format

Parameters: temp - Temperature, buffer - Output

Action: Calls float_to_string(), appends ‘C’

```

static void temp_to_string(float temp, char *buffer)
{
    float_to_string(temp, buffer, 1); /* 1 decimal place */

    /* Find end of string */
    while (*buffer != '\0') {
        buffer++;
    }

    /* Add 'C' suffix */
    *buffer++ = 'C';
    *buffer = '\0';
}

```

11. SYSTEM CONSTANTS

Buzzer Tones

330 Hz (NOTE_E4): Entry/Exit, Neutral tone
 523 Hz (NOTE_AFFIRMATIVE): Success confirmation
 165 Hz (NOTE_ALERT): Warning/Access denied
 1047 Hz (NOTE_EARTHQUAKE): Critical earthquake alert

Motion Thresholds

0-10: No movement
 10-35: Minor movement (alert beeps)
 35-60: Significant movement

>60: Earthquake condition

Temperature Steps

20°C: 0%
20-21°C: 25%
21-22°C: 50%
22-23°C: 75%
>23°C: 100%

Timing Parameters

Door open timeout: 15 seconds (safety)
IR detection delay: 5 seconds (no-motion close)
Path clear wait: 3-5 seconds
LCD update cycle: 2 seconds per state
Main loop delay: 500 ms

HARDWARE CONNECTIONS SUMMARY

Component	Pin	Type	Interface	Function
LM35	PA0	Analog	ADC1	Temperature input
Buzzer	PA15	PWM	TIM2_CH1	Frequency sound
Servo	PB0	PWM	TIM3_CH3	Door lock (0-180°)
I2C LCD	PB6-7	I2C	I2C1	Display output
Accelerometer	PA5-7, PE3	SPI	SPI1	Motion detection
RFID Module	PB13-15, PD8-9, PC6	SPI	SPI2	Card reading
Fan	PD15	PWM	TIM4_CH4	Speed control
IR Sensor	PC7	GPIO IN	GPIOC	Door passage detect
Exit Button	PC8	GPIO IN	GPIOC	Exit request
LED Light	PC9	GPIO OUT	GPIOC	Room light
LED OFF Button	PE6	GPIO IN	GPIOE	Light toggle

COMPILATION & TESTING

Build Steps

```
# Compile all sources
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -O2 -c *.c

# Link
arm-none-eabi-ld -o firmware.elf *.o
```


Create HEX

```
arm-none-eabi-objcopy -O ihex firmware.elf firmware.hex
```

Testing Checklist

- ☐ RFID card detection & UID reading
 - ☐ Temperature reading (should update every ~2 seconds)
 - ☐ Fan speed ramping with temperature
 - ☐ Servo angle control (0° locked, 180° open)
 - ☐ Buzzer tone verification
 - ☐ LCD display initialization & text
 - ☐ Light toggle via PE6 button
 - ☐ Motion detection sensitivity
 - ☐ Entry/Exit sequences
 - ☐ Door timeout on no-motion
-

VIVA QUESTIONS & DETAILED EXPLANATIONS

QUESTION 1: RFID Module - Registers, Anti-Collision, and Pin Functions

Q1.1: Why So Many Registers in RC522?

Answer: The RC522 RFID reader needs multiple registers to control different aspects of the communication protocol. Each register manages a specific function:

Register Breakdown and Purpose:

Register	Address	Purpose	Why Needed
CommandReg	0x01	Controls current command (Idle, Transceive, Reset)	Execute different RFID operations
ComIrqReg	0x04	Interrupt request flags (RxIRq, TxIRq, etc.)	Detect when data is ready or sent
FIFODataReg	0x09	Data buffer (send/receive)	Queue data to transmit or read received data
FIFOLevelReg	0x0A	How many bytes in FIFO	Know how much data is waiting to be read

Register	Address	Purpose	Why Needed
ControlReg	0x0C	Control bits (LastBits, TStop, etc.)	Fine-tune bit timing and reception
BitFramingReg	0x0D	Bit framing control (TxLastBits, RxAlign)	Handle partial byte transmission
ErrorReg	0x06	Error flags (ProtErr, BufferOvfl, CRCErr)	Detect transmission failures
ModeReg	0x11	Transceiver mode configuration	Set modulation and timing parameters
TxASKReg	0x15	Transmit Amplitude Shift Keying	Control TX power level
TxControlReg	0x14	TX1/TX2 output enable	Enable antenna outputs
VersionReg	0x37	Hardware version ID	Verify correct chip is present

Why This Design? - **Modularity:** Each register controls one feature, making debugging easier - **Precision:** Fine-grained control over timing and power - **Flexibility:** Different card types require different settings - **Real-time Monitoring:** Interrupt registers allow event-driven programming

Q1.2: What is the Anti-Collision Algorithm?

Answer: Anti-collision is a mechanism that allows the reader to identify and communicate with only ONE card when multiple cards are present in the field simultaneously.

How Anti-Collision Works:

SCENARIO: 3 RFID cards in range simultaneously

1. READER sends REQA (Request Type A)
↓
2. ALL CARDS respond simultaneously (would cause collision)
↓
3. READER sends ANTICOLL command with UID position selector
↓
4. Cards compare their UID bits:
 - If bit matches selector: transmit "0"
 - If bit doesn't match: stay silent

- ↓
5. READER reads response, identifies one card bit-by-bit
 - ↓
 6. READER selects that specific card
 - ↓
 7. OTHER CARDS shut up, only selected card responds
 - ↓
 8. Process repeats for next card in range

Step-by-Step Example:

Suppose 3 cards with UIDs:

Card A: 0x63954E56

Card B: 0xD06E6D32

Card C: 0x12AB34CD

Step 1: Reader sends ANTICOLL (0x93) + Bit selector (0x20 = all 40 bits)

Step 2: Cards respond with their bits:

Card A bit 0 = 0 (from 0x63...)

Card B bit 0 = 1 (from 0xD0...)

Card C bit 0 = 0 (from 0x12...)

Result: Collision (0 and 1 both present)

Step 3: Reader tries ANTICOLL with selector LSB bit = 0

Only Cards A and C respond (they have bit 0 = 0)

Step 4: Reader tries ANTICOLL with selector LSB bit = 1

Only Card B responds (it has bit 0 = 1)

Step 5: Reader selects Card B (0xD06E6D32)

Cards A and C stop responding

Step 6: Reader can now repeat for Cards A and C

Implementation in Code:

```
/* ANTICOLL command structure */
MFRC522_WriteReg(FIFODataReg, PICC_ANTICOLL); // 0x93 = Anti-collision instruction
MFRC522_WriteReg(FIFODataReg, 0x20);          // 0x20 = Select all bits

/* After IRQ, read 5 bytes */
for (int i = 0; i < 5; i++)
    uid[i] = MFRC522_ReadReg(FIFODataReg); // 4 UID bytes + 1 checksum
```

Why Anti-Collision Matters: - Multiple cards may be in range accidentally - Standard protocol requirement (ISO 14443A) - Prevents “collision” noise that corrupts data - En-

sure only one card is selected at a time - Allows sequential card reading in multi-card scenarios

Q1.3: RFID Module Pin Functions

Pin Configuration (RC522):

PIN	Type	Purpose	Connected To
VCC	POWER	+3.3V Supply	STM32 3.3V
GND	POWER	Ground	STM32 GND
SDA/MOSI	DATA	Serial Data In (SPI Master→Slave)	PB15 (SPI2_MOSI)
SCK	CLK	SPI Clock from Master	PB13 (SPI2_SCK)
SDO/MISO	DATA	Serial Data Out (Slave→Master)	PB14 (SPI2_MISO)
SS/NSS	CTRL	Chip Select (Active Low)	PD8 (GPIO, manual control)
RST/RESET	CTRL	Hardware Reset (Active Low)	PD9 (GPIO, manual control)
IRQ	INT	Interrupt Output (Active Low)	PC6 (EXTI9_5)

Pin-by-Pin Explanation: **1. SDA/MOSI (Pin 4) - Full Name:** Serial Data In / Master Out Slave In - **Function:** Master (STM32) sends commands/data to RC522 - **Signal:** Rising/falling edges represent 0/1 bits - **Why Needed:** RC522 must receive instructions and data from microcontroller - **Connected:** PB15 → SPI2_MOSI

2. SCK (Pin 7) - Full Name: Serial Clock - **Function:** Synchronizes data transmission - **Frequency:** ~5.25 MHz (controlled by prescaler) - **Why Needed:** SPI is synchronous; clock tells slave when to read/write - **Connected:** PB13 → SPI2_SCK

3. SDO/MISO (Pin 6) - Full Name: Serial Data Out / Master In Slave Out - **Function:** RC522 sends response data back to STM32 - **Signal:** Register values, FIFO data, interrupt flags - **Why Needed:** STM32 must read card UIDs and status information - **Connected:** PB14 → SPI2_MISO

4. SS/NSS (Pin 24) - Full Name: Slave Select / Chip Select - **Function:** Activates RC522 for communication (Active Low) - **Typical Signal:** HIGH (inactive): RC522 ignores SPI bus LOW (active): RC522 listens to SPI commands - **Why Needed:** Allows one STM32 to control multiple SPI devices - **Usage Pattern:**

```
c  RFID_CS_LOW();           // Start communication  HAL_SPI_Transmit(...) // Send
data  HAL_SPI_Receive(...) // Receive response  RFID_CS_HIGH();           // End
communication - Connected: PD8 → GPIO_PIN_8 (manually controlled)
```

5. RST/RESET (Pin 11) - Full Name: Hardware Reset - **Function:** Forces RC522 into initial state (Active Low) - **Reset Sequence:** Normal: RST = HIGH (working) Reset: RST = LOW (100ps minimum) Recovery: RST = HIGH (wait 50ms) - **Why Needed:** Clear all registers, reinitialize chip after power-up - **Usage in Code:**

```
c  RFID_RST_HIGH(); // Normal operation // At startup or
if stuck:  RFID_RST_LOW(); // Hold reset  HAL_Delay(1); // 1ms minimum
RFID_RST_HIGH(); // Release reset  HAL_Delay(50); // Wait for chip init - Connected: PD9 → GPIO_PIN_9 (manually controlled)
```

6. IRQ (Pin 13) - Full Name: Interrupt Request - **Function:** Signals event completion (Active Low) - **Typical Events:** - Card detected (ATQA received) - UID collision resolved - Data transmission complete - Error condition (parity error, timeout) - **Timing:** Pulse from HIGH → LOW when event occurs - **Why Needed:** No need to continuously poll; interrupt signals when data ready - **Usage Pattern:** `/* Without IRQ (polling): */
while (n < 1000) { // Waste CPU waiting uint8_t irq = MFRC522_ReadReg(ComIrqReg);
if (irq & 0x30) break; // Check manually n++; HAL_Delay(1); }

/* With IRQ (event-driven): */ // Hardware fires interrupt automatically void
EXTI9_5_IRQHandler(void) { // Process card detection immediately } */` - **Connected:** PC6 → EXTI9_5 (Interrupt with falling edge trigger)

Q1.4: Complete RFID Communication Example

SEQUENCE: Reading a card UID

1. MFRC522_Request() - Card detection
 - Send REQA (0x26) command
 - Wait for ATQA response (2 bytes)
 - Validate no errors
 2. MFRC522_Anticoll() - Get unique UID
 - Send ANTICOLL (0x93) command
 - Add 0x20 (select 40 bits)
 - Read 5-byte response (4 UID + checksum)
 - Validate checksum & bit alignment
 3. Card UID extracted: 0xD06E6D32
 - Can now check against whitelist
-

QUESTION 2: I2C LCD Module - Why Only 2 Wires?

Q2.1: What is I2C and Why Use It?

I2C (Inter-Integrated Circuit) is a serial communication protocol that uses only **2 wires** to communicate with multiple devices.

Traditional LCD (Parallel) vs I2C LCD:

PARALLEL LCD (8-bit mode):

STM32	D0	LCD
	D1	LCD
	D2	LCD
GPIO	D3	LCD

Pins	D4	LCD
	D5	LCD
	D6	LCD
	D7	LCD
	RS	LCD
	RW	LCD
	EN	LCD

Total: 11 pins needed

I2C LCD (with PCF8574 I2C expander):

STM32	PCF8574	LCD
I2C	I2C Expander	→ 16x2
SDA		
SCL	P0-P7 ports	

Total: 2 pins needed (SDA, SCL)

Advantages of I2C: - Only 2 wires (scalable to 127 devices) - Reduced pin count on microcontroller - Built-in I2C module faster than bit-banging - Perfect for space-constrained projects

Q2.2: How Does I2C Work?

Basic Concept:

SDA (Serial Data Line): Carries actual data

SCL (Serial Clock Line): Synchronizes communication

Both pulled HIGH by resistors (open-drain design)

Either device can pull LOW (wired-AND)

Frame Structure:

START Address (7 bits) R/W ACK Data Bytes ACK STOP

Example: Writing to LCD via I2C

/ Sending "A" to LCD row 0, col 0 */*

1. START condition: SDA HIGH→LOW **while** SCL HIGH

SDA

SCL

2. Send address `0x27` (PCF8574) + WRITE (0)
Bits: `0010011|0`
 3. Master pulls SDA LOW **for** bit=0
Master pulls SDA HIGH **for** bit=1
Master pulses SCL to latch each bit
 4. PCF8574 acknowledges (ACK)
 5. Send 4-byte control sequence
Byte 1: `0x80 + 0x00 = 0x80` (Set cursor row 0, col 0)
Byte 2: `0x41` (ASCII 'A')
Byte 3: Control bits (EN pulse high)
Byte 4: Control bits (EN pulse low)
 6. STOP condition: SDA LOW→HIGH **while** SCL HIGH
-

Q2.3: PCF8574 I2C Expander - Bridging the Gap

Why This Module Works:

The PCF8574 is an **8-bit I/O expander** that converts: - **2-wire I2C input → 8-port GPIO output**

PCF8574 Pin Layout:

I2C Side (to STM32): GPIO Side (to LCD):

VCC → 3.3V		P0 → D4 (LCD)
GND → GND		P1 → D5 (LCD)
SDA → PB7 (I2C)	→	P2 → D6 (LCD)
SCL → PB6 (I2C)	→	P3 → D7 (LCD)
A0,A1,A2 → GND		P4 → EN (LCD)
(Address bits)		P5 → RS (LCD)
	P6 → BL (backlight)	
	P7 → NC	
	→	

How PCF8574 Transfers Data:

STM32 sends via I2C: Single byte (0xBD)
Binary: 1011 1101

PCF8574 outputs: P0=1, P1=0, P2=1, P3=1
P4=1, P5=1, P6=0, P7=1

LCD receives: D4=1, D5=0, D6=1, D7=1
 EN=1, RS=1, BL=0, NC=1

Q2.4: Why I2C LCD in This Project?

Space Constraints: - STM32F407 has limited GPIO pins when using SPI1, SPI2, ADC, timers - Original parallel LCD needs 11 pins - I2C LCD needs only 2 pins

Pin Count Analysis:

Used Peripherals:

SPI1 (Accel): PA5, PA6, PA7, PE3 = 4 pins
SPI2 (RFID): PB13, PB14, PB15, PD8, PD9, PC6 = 6 pins
TIM2 (Buzzer): PA15 = 1 pin
TIM3 (Servo): PB0 = 1 pin
TIM4 (Fan): PD15 = 1 pin
GPIO (Light, Buttons): PC7, PC8, PC9, PE6 = 4 pins
I2C (LCD): PB6, PB7 = 2 pins

Total: 19 pins (vs 30 with parallel LCD)

Q2.5: Complete I2C LCD Operation

Initialization Sequence:

```
void lcd_init(void) {  
    HAL_Delay(50);           // Wait for power stabilization  
  
    lcd_send_cmd(0x30);      // 8-bit mode initialization  
    lcd_send_cmd(0x02);      // Return home  
    lcd_send_cmd(0x28);      // 4-bit mode, 2 lines, 5x8 font  
    lcd_send_cmd(0x0C);      // Display ON  
    lcd_send_cmd(0x06);      // Auto-increment cursor  
    lcd_send_cmd(0x01);      // Clear display  
  
    HAL_Delay(2);  
}
```

Data Transmission Example:

```
/* Sending 'A' via I2C to LCD */  
  
lcd_send_data('A');  
  
↓ (internally)
```



```
uint8_t data = 'A' = 0x41 = 01000001
```

Split into nibbles:

```
High (0x40): 0100----
```

```
Low  (0x10): 0001----
```

Control bits pattern:

```
Bit 3: Enable (EN)
```

```
Bit 2: RS (0=cmd, 1=data)
```

```
Bit 1: RW (0=write)
```

```
Bit 0: Backlight
```

Transmission sequence:

```
1. High nibble + EN=1, RS=1 (data), BL=1
```

```
Send: 0x4D (01001101) via I2C
```

```
2. Enable pulse
```

```
3. High nibble + EN=0
```

```
Send: 0x49 (01001001) via I2C
```

```
4. Low nibble + EN=1, RS=1
```

```
Send: 0x1D (00011101) via I2C
```

```
5. Enable pulse
```

```
6. Low nibble + EN=0
```

```
Send: 0x19 (00011001) via I2C
```

Result: LCD displays 'A'

QUESTION 3: Why DMA for ADC Instead of Other Options?

Q3.1: ADC Data Collection Methods

There are **4 main approaches** to read ADC data:

Method 1: Blocking/Polling (No DMA)

```
while(1) {
    HAL_ADC_Start(&hadc1);                // Start conversion
    HAL_ADC_PollForConversion(&hadc1, 1000); // Wait for completion
    uint32_t value = HAL_ADC_GetValue(&hadc1); // Read value
    HAL_ADC_Stop(&hadc1);                  // Stop

    // Process temperature
    float temp = (value * 3.3f / 4095.0f) * 100.0f;

    HAL_Delay(10); // Sample every 10ms
}
```

Pros: - Simple, easy to understand - No additional hardware setup

Cons: □ - **CPU BLOCKS** entire loop waiting for ADC - Other tasks (buzzer, servo, LED) freeze during ADC - Inefficient, wastes CPU cycles - 5.9μs conversion time × repeated polling = lots of idle waiting - **Performance Impact:** Main loop could be blocked for 100+ μs per sample

Method 2: Interrupt-Driven (No DMA)

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc) {
    if (hadc->Instance == ADC1) {
        adc_value = HAL_ADC_GetValue(hadc);
        // Process temperature immediately
        float temp = (adc_value * 3.3f / 4095.0f) * 100.0f;
    }
}

// In main loop:
while(1) {
    HAL_ADC_Start_IT(&hadc1); // Start with interrupt

    // Do other tasks while ADC runs
    led_toggle();
    buzzer_play(330);
    servo_set_angle(90);

    HAL_Delay(10);
}
```

Pros: - □ Non-blocking, main loop continues - □ Interrupt fires on conversion complete
- □ Better than polling

Cons: □ - CPU still gets interrupted ~100 times/sec - **Context switching overhead:** Save/restore registers (~20 CPU cycles each) - Can interfere with time-critical operations (PWM, precise servo control) - Only moves data once per sample (~5 bytes per interrupt) - **Performance Impact:** 100 interrupts/sec × 20 cycles = 2000 CPU cycles wasted on context switching

Method 3: DMA (Recommended for This Project) □

```
/* Initialization */
HAL_ADC_Start_DMA(&hadc1, (uint32_t *)&adc_value, 1);

// In main loop (no ADC code needed):
while(1) {
    float temp = LM35_GetTemperature(); // Reads DMA buffer directly

    // CPU fully available for other tasks
}
```

```

    led_toggle();
    buzzer_play(330);
    servo_set_angle(90);

    HAL_Delay(10);
}

```

How DMA Works:

1. ADC conversion completes
2. ADC triggers DMA (not CPU)
3. DMA transfers data directly: ADC_DR → adc_value
4. No CPU interrupt, no context switching
5. Main program continues uninterrupted
6. Software reads adc_value whenever ready

Pros: ☐☐☐ - **Zero CPU Overhead** - No interrupts, no blocking - **Automatic continuous sampling** - Circular mode - **High efficiency** - DMA is hardware, not software - **Non-intrusive** - Main loop not affected at all - **Better real-time performance** - PWM, servo timing precise - **Scalable** - Could add multiple ADC channels easily - **Performance Impact:** ZERO CPU cycles wasted

Cons: - Slightly more complex setup - Requires understanding DMA concepts

Method 4: ADC with Watchdog (Rarely Used)

```

/* Triggers interrupt only if value exceeds threshold */
HAL_ADC_Start_IT_with_Watchdog(&hadc1, 3000); // Alert if > 3000

```

Only useful if you only care about threshold violations, not continuous values.

Q3.2: Comparison Table

Aspect	Polling	Interrupt	DMA	Watchdog
CPU Overhead	High	Medium	None	Low
Non-blocking	No	Partial	Yes	Yes
Context Switching	No	Yes	No	Yes
Sampling Frequency	Limited	100+ Hz	Unlimited	Limited
Continuous Data	Manual	ISR-based	Automatic	Events only

Aspect	Polling	Interrupt	DMA	Watchdog
Real-time Performance	Poor	Fair	Excellent	Good
Complexity	Simple	Medium	Medium	Complex
Best For	Testing	Non-critical	Continuous	Alarms

Q3.3: DMA Configuration for This Project

```

/* DMA Setup */
static void MX_DMA_Init(void) {
    __HAL_RCC_DMA2_CLK_ENABLE();

    hdma_adc1.Instance = DMA2_Stream0;
    hdma_adc1.Init.Direction = DMA_PERIPH_TO_MEMORY;
    hdma_adc1.Init.PeriphInc = DMA_PINC_DISABLE;    // Don't increment ADC address
    hdma_adc1.Init.MemInc = DMA_MINC_DISABLE;       // Don't increment adc_value address
    hdma_adc1.Init.Mode = DMA_CIRCULAR;             // Continuous loop
    hdma_adc1.Init.Priority = DMA_PRIORITY_HIGH;    // High priority (temperature is important)

    HAL_DMA_Init(&hdma_adc1);
    __HAL_LINKDMA(&hadc1, DMA_Handle, hdma_adc1); // Connect to ADC
}

/* Usage */
void LM35_StartConversion(void) {
    HAL_ADC_Start_DMA(&hadc1, (uint32_t *)&adc_value, 1);
    // DMA now runs forever, updating adc_value automatically
}

```

Why This Configuration? - DMA_PERIPH_TO_MEMORY: ADC data flows to RAM - DMA_MINC_DISABLE: Always write to same location (adc_value) - DMA_CIRCULAR: Never stops, continuous sampling - HIGH_PRIORITY: Temperature control is critical for safety

Q3.4: Real-World Performance Impact

Scenario: Temperature monitoring + Buzzer playing + Servo control

WITH POLLING:

Time	CPU Operation
0 µs	ADC_Start()
1 µs	[BLOCKING] Polling...

```

6 µs    ADC complete
6 µs    Get value
7 µs    Calculate temperature
9 µs    [Now can do other stuff]
9-10000 µs  Buzzer, Servo, etc.

```

Block time: ~7 µs per sample
Samples/sec: 100 Hz = 700 µs/sec wasted = 0.07% CPU

PROBLEM: If you have 10 ADC channels or multiple tasks block,
CPU usage explodes and real-time tasks suffer.

WITH DMA (This Project):

```

Time      CPU Operation

0 µs      ADC_Start_DMA()
1 µs      CPU continues freely      ← ADC+DMA running in background
6 µs      Buzzer playing
6 µs      Servo positioning
6 µs      Light control
6 µs      All tasks running
6 µs      [DMA updates adc_value] ← Invisible to CPU
10000 µs  Read temperature when ready

```

CPU wasted: 0 µs
Overhead: < 0.1% CPU
Real-time guarantee: Excellent

QUESTION 4: Communication Protocols Used - General Explanation

Q4.1: Protocol Overview Table

Protocol	Purpose	Devices	Pins	Speed	Sync
SPI	Fast serial	Accel, RFID	4+	~5 MHz	Yes
I2C	General I/O	LCD	2	100 kHz	Yes
PWM	Motor/Audio	Buzzer, Fan, Servo	1	50 Hz	Yes
ADC	Analog input	Temperature	1	21 MHz	No
GPIO	Digital I/O	Buttons, LED	1+	N/A	No

Q4.2: Detailed Protocol Explanations

1. SPI (Serial Peripheral Interface) What is SPI? High-speed synchronous serial protocol for close-range communication (same board).

Architecture:

MASTER (STM32)	SLAVE 1 (RFID)	SLAVE 2 (Accel)
SCK	SCK	SCK
MOSI	MOSI	MOSI
MISO	MISO	MISO
CS1	CS	
CS2		CS

Principle: - **Clock (SCK):** Master provides clock (synchronous) - **MOSI:** Master sends data to slave - **MISO:** Slave sends data to master - **CS (Chip Select):** Activates specific slave

Timing:

SCK:
(8 clock cycles)

MOSI:
1 0 1 1 0 1 0 0 (0xB4)
(each bit sampled on clock edge)

MISO:
0 1 1 1 1 0 0 1 (0x79)
(shift in response simultaneously)

Usage in This Project:

SPI1 - Accelerometer (LIS302DL):

```
ACC_CS_LOW(); // Select accelerometer
HAL_SPI_Transmit(&hspi1, &cmd, 1, HAL_MAX_DELAY); // Send command
HAL_SPI_Receive(&hspi1, &data, 1, HAL_MAX_DELAY); // Read response
ACC_CS_HIGH(); // Deselect
```

SPI2 - RFID (RC522):

```
RFID_CS_LOW();
HAL_SPI_Transmit(&hspi2, cmd, 2, HAL_MAX_DELAY);
HAL_SPI_Receive(&hspi2, response, 1, HAL_MAX_DELAY);
RFID_CS_HIGH();
```

Why SPI for These Devices? - □ High speed (~5 MHz) needed for RFID/Accel - □ Short distance (same PCB) - □ Multiple devices with separate CS pins - □ Not for long

cables (noise susceptible)

SPI Modes (CPOL, CPHA):

Mode 0: CPOL=0, CPHA=0 (Clock low idle, sample on rising edge)
Mode 1: CPOL=0, CPHA=1 (Clock low idle, sample on falling edge)
Mode 2: CPOL=1, CPHA=0 (Clock high idle, sample on falling edge)
Mode 3: CPOL=1, CPHA=1 (Clock high idle, sample on rising edge)

This project uses:

- SPI1 (Accel): Mode 0 (CPOL=0, CPHA=0)
 - SPI2 (RFID): Mode 1 (CPOL=0, CPHA=1) ← Different!
-

2. I2C (Inter-Integrated Circuit) What is I2C? Low-speed synchronous bus with addressable slaves, supports multiple devices on 2 wires.

Architecture:

SDA (Data)

STM32 PCF8574(0x27) LCD EEP Other

SCL (Clock)

Pull-up resistors (4.7k each)

Principle: - **Open-drain design:** Devices pull low, resistors pull high - **Addressable:** Each device has unique address (0-127) - **Collision handling:** Devices check if line is free - **Synchronous:** Clock synchronizes data

Frame Structure:

START	Address (7 bits)	R/W	ACK	Data Bytes	ACK	STOP
	(0x27 for LCD)	(0=W)	Slave	(variable)		

START: SDA: HIGH→LOW while SCL HIGH

SDA

SCL

STOP: SDA: LOW→HIGH while SCL HIGH

SDA

SCL

Example: Writing to LCD

1. START: Pull SDA low while SCL high
2. Address: Send 0x27 (PCF8574) = 0010011 in binary
3. R/W bit: Send 0 (write)
4. ACK: PCF8574 pulls SDA low for 1 clock
5. Data: Send control byte (e.g., 0xBD)
6. ACK: Receive
7. STOP: Release SDA while SCL high

Waveform:

START	0	0	1	0	0	1	1	[W]	[ACK]	...
	↓	↓	↑	↓	↓	↑	↑			

SDA:

...

SCL:

Clock Stretching:

Master sends clock, but slave can hold SCL low to pause:

SCL (Normal):

SCL (With hold):

^Slave pulls SCL low

Why I2C for LCD? - □ Only 2 wires (vs 8+ for parallel) - □ Can add more devices (EEPROM, sensors) - □ Long cable support (up to 2 meters) - □ Lower power - □ Slower (100 kHz vs 5 MHz)

3. PWM (Pulse Width Modulation) What is PWM? Varying ON/OFF time of a digital signal to control analog devices (speed, brightness, frequency).

Principle:

Frequency = Base_Clock / (Prescaler × Period)

Duty = (Pulse / Period) × 100%

BUZZER (variable frequency):

Freq = 1,000,000 Hz / (Period + 1)

330 Hz: Period = 3030, Pulse = 1515 (50%)

523 Hz: Period = 1911, Pulse = 955 (50%)

Waveform:

330 Hz

↑(ON:1515)↓(OFF:1515)

FAN (fixed frequency, variable duty):

Freq = 50 Hz (fixed)

Period = 20000 (20 ms period)

25% speed: Pulse = 5000 (5 ms ON, 15 ms OFF)

50% speed: Pulse = 10000 (10 ms ON, 10 ms OFF)

100% speed: Pulse = 20000 (20 ms ON, 0 ms OFF)

25%

50%

100%

Timer Configuration for Each Device:

TIM2 (Buzzer):

Prescaler: 84 - 1 = 83 // Clock: 1 MHz
Period: Dynamic // Frequency varies
Duty: 50% // Square wave

TIM3 (Servo):

Prescaler: 84 - 1 // Clock: 1 MHz
Period: 20000 - 1 // Freq: 50 Hz (20 ms)
Pulse: 1000-2000 // 1-2 ms pulse width
0°: 1000 µs (locked)
90°: 1500 µs (neutral)
180°: 2000 µs (unlocked)

TIM4 (Fan):

Prescaler: 84 - 1 // Clock: 1 MHz
Period: 20000 - 1 // Freq: 50 Hz (20 ms)
Pulse: 0-20000 // 0-100% duty

0: 0% (off)
5000: 25% speed
10000: 50% speed
20000: 100% speed

Why PWM? - □ Analog control with digital signal - □ Efficient power delivery - □ Micro-controller can't output true analog - □ Devices react to average power

4. ADC (Analog-to-Digital Converter) What is ADC? Converts continuous analog voltage (0-3.3V) to digital value (0-4095).

Architecture:

LM35 Sensor	ADC Module	Digital Value
0°C = 0.00V	→	0
10°C = 0.10V	→	125
20°C = 0.20V	→	246
30°C = 0.30V	→	369
50°C = 0.50V	→	614
100°C = 1.00V	→	1229

Formula:

$\text{ADC_Value} = (\text{Voltage} / 3.3\text{V}) \times 4095$
 $\text{Temperature} = \text{ADC_Value} \times 330 / 4095$

ADC Configuration:

Resolution: 12-bit (0-4095)
Reference: 3.3V
Channel: ADC1_IN0 (PA0)
Sampling: 112 cycles (slower = more stable)
Clock: 21 MHz (84 MHz / 4)
Conversion: ~5.9 µs per sample
Frequency: ~170 samples/sec max (without DMA)

Timing Diagram:

Start Conversion

Sampling Period 112 cycles @ 21 MHz = 5.3 µs
(11 µs typical)

Conversion 12 cycles = 0.57 µs
Period

DATA READY (total ~5.9 μ s)

Why DMA for ADC? As discussed in Question 3 - zero CPU overhead, continuous sampling.

5. GPIO (General Purpose I/O) What is GPIO? Simple digital pins that are either HIGH (3.3V) or LOW (0V).

Usage in This Project:

Input Pins (Read):

PC7: IR Sensor (detects person)
PC8: Exit Button (pull-up, active-low)
PE6: LED OFF Button (pull-up, active-low)

Output Pins (Write):

PC9: LED Light (HIGH=ON, LOW=OFF)
PE3: Accel CS (HIGH=inactive, LOW=active)
PD8: RFID CS (HIGH=inactive, LOW=active)
PD9: RFID RST (HIGH=normal, LOW=reset)

GPIO Timing:

Reading button:

(Released)

(Pressed)

↑ Read (HIGH) ↑ Read (LOW)

Debounce delay: 300 ms minimum

Q4.3: Protocol Comparison - When to Use Which?

Task	Protocol	Reason
Temperature sensor	ADC	Analog voltage from LM35
RFID card reading	SPI2 @ 5 MHz	High speed, short range
Accelerometer data	SPI1 @ 5 MHz	High speed, short range
LCD display	I2C @ 100 kHz	Only 2 wires, scalable
Buzzer frequency	PWM / TIM2	Variable tone generation
Servo positioning	PWM / TIM3	Pulse width = angle

Task	Protocol	Reason
Fan speed	PWM / TIM4	Duty cycle = speed
Push buttons	GPIO input	Digital state only
LED light	GPIO output	Simple ON/OFF

Q4.4: Protocol Layering in This System

Application Layer

Room Access Control
 Temperature Management
 Motion Alarms
 User Interaction

Hardware Abstraction Layer (HAL)

lcd_print() → I2C
 ACC_ReadAxis() → SPI1
 MFRC522_ReadReg() → SPI2
 LM35_GetTemp() → ADC
 buzzer_play() → PWM/TIM2
 servo_set_angle() → PWM/TIM3
 fan_set_speed() → PWM/TIM4
 light_on() → GPIO

Protocol Layer

SPI (Accel, RFID)
 I2C (LCD)
 PWM (Buzzer, Servo, Fan)
 ADC (Temperature)
 GPIO (Buttons, LED)

STM32F407 Hardware

SPI1, SPI2 Controllers
 I2C1 Controller
 TIM2, TIM3, TIM4 (PWM)
 ADC1 + DMA2
 GPIO Ports A-E

Q4.5: Clock Speed Requirements

Protocol | Clock Speed | Why?

SPI1 (Accel)		5.25 MHz	Fast sensor data ~100Hz
SPI2 (RFID)		5.25 MHz	Fast protocol required
I2C (LCD)		100 kHz	Simple display, no speed needed
ADC (Temp)		21 MHz	Conversion clock (input)
PWM (Buzzer)		1 MHz	Frequency calculation
PWM (Servo)		1 MHz	Precise angle control
PWM (Fan)		1 MHz	Smooth speed ramping
